

Project Design and Mockups

Project / App Name: OnboardBuddy

Team Name: OnboardBuddies

Team Members:

Nam Le — DevOps / Database / Security

Bradley Sakran — Backend / Analysis Engine

Sahib Rao — Backend / UI/UX / Ranking

Eugene Ng — Frontend / QA

1. Project Summary

1.1 Project Description

OnboardBuddy is a codebase onboarding platform that generates and maintains a **Codebase Onboarding Package** for a repository. It focuses on helping developers understand a specific codebase faster and become productive without hours of manual code reading or repeated explanations.

OnboardBuddy's core differentiator is its **deterministic analysis pipeline**. OnboardBuddy builds a structured code evidence model from source code using the TypeScript Compiler API, extracts workflows algorithmically, ranks critical learning paths using a weighted composite scoring system, and then produces persistent, versioned, role-specific onboarding documentation grounded in verified code references.

The LLM layer is used as an optional explanation layer on top of already-extracted, validated structure. The app calls OpenRouter through an OpenAI-compatible API shape, with OpenAI as the default configured provider. Substantive generated explanations are tied to source receipts where possible and assigned a confidence label. If a statement cannot be directly grounded, it will be marked as low confidence or omitted from the approved package. Generated content is treated as a draft until a team reviewer approves it.

1.2 Inspirations

Claude Code / Cursor / Windsurf

Borrowed:

- AI-assisted code explanation
- Repository-aware reasoning
- Developer productivity focus

Difference:

- They are chat-first; OnboardBuddy is onboarding-package-first.
- They answer questions in sessions; OnboardBuddy stores, updates, and validates onboarding knowledge over time.
- Their outputs are short-lived; OnboardBuddy's outputs are persistent, versioned, and reviewable.
- Their understanding is generated; OnboardBuddy's understanding is extracted deterministically and then explained.

ScopeDocs

Borrowed (suggested by peer feedback — David Falade):

- Read-only OAuth trust model
- No full hosted copy of codebase by default
- Index metadata and needed code slices
- Permissions mirror GitHub access
- Generated claims tied to source receipts

Difference:

- ScopeDocs generates documentation.
- OnboardBuddy generates codebase onboarding packages with interactive walkthroughs, ranked learning paths, and incremental stale-section updates.

Existing Engineering Onboarding Docs

Borrowed:

- "Start here" repo overview
- Local setup instructions
- Request lifecycle explanation
- Architecture references
- Testing and safety scripts

Improvement:

- Generated from repository evidence, not written manually
- Updated through git diffs
- Role-specific
- Source-grounded
- Pull-based, not overwhelming

1.3 Target Users

User	What They Use OnboardBuddy For
New developer	Understand the repo structure, find first files to read, and role-specific code
Intern / co-op	Become productive without waiting for repeated explanations
Returning maintainer	See what changed and which onboarding sections are stale
Engineering lead	Review generated onboarding drafts instead of writing from scratch
Handoff team	Produce a handoff package for ownership transfer

1.4 Data

Data Collected

OnboardBuddy collects data from two sources: the user and their GitHub account.

From users directly:

- Email address (via GitHub OAuth sign-in through Supabase Auth)
- Developer role selection (backend / frontend / devops / qa / general) when joining a project
- Project configuration choices: target repository, branch, ignored paths, privacy notice acceptance
- Invitation recipient email addresses (when inviting team members)
- Permission tier assignments (owner / admin / developer)

From GitHub (via GitHub OAuth and GitHub App):

- GitHub user ID and username
- OAuth access token (stored encrypted) and granted scopes
- Repository metadata: owner, name, available branches, default branch
- Repository source archive (temporary zipball download for analysis, discarded after)
- Commit history and diffs for change detection

Data Used During Analysis

OnboardBuddy uses repository and commit-history data:

- Source files, especially TypeScript / TSX
- Config files, including package, TypeScript, framework, CI/CD, Docker, extension, and serverless configs
- Database schema files, migrations, ORM models, API schemas, shared domain types
- Tests, coverage reports, fixtures
- Git metadata: commits, diffs, file churn, changed files, branch, analyzed commit hash
- (Stretch) README, docs, ADRs (Architecture Design Record), JSDoc (Function comments), inline comments

Data Stored

User and project data:

- User accounts: ID, email, account creation time
- GitHub connections: GitHub user ID, username, encrypted OAuth token, granted scopes
- Projects: repository owner, name, branch, ignored paths, privacy notice status, analysis state
- Project members: permission tier, developer role, join time
- Project invitations: recipient email, assigned permission tier, optional developer role, invitation status (pending / accepted / revoked / expired), expiration time

Analysis artifacts:

- Analysis snapshots: commit hash, branch, trigger type (manual / re-analysis), job status, timing
- File-level evidence: file paths, file hashes, language, category, line counts
- Symbol-level evidence: symbol names, kinds, positions, AST-derived metadata, symbol hashes
- Module dependency graph edges: source symbol, target symbol, relationship type (import, call, extend, implement)
- Workflow candidates: entry symbol, traced path, classification, scores
- Critical path rankings: workflow scores, role-weighted priorities, Critical 25%
- Generated onboarding package sections: role, section type, content, source receipts, review/approval status
- Interactive walkthrough steps: sequence, target files and symbols, explanation content, source receipts
- Source receipts: file path, symbol, line range, commit hash (linking every generated claim to code)
- AI generation receipts (Stretch): prompt hash, model, token usage, latency (for cost tracking and auditability)

Default storage policy:

- Do **not** store a full permanent copy of the repo by default.
- Store metadata, graph relationships, hashes, generated docs, and selected cited snippets.
- Temporary archive extraction may be used during analysis and discarded after.
- Persistent storage keeps only what is needed to maintain and verify the onboarding package.

Privacy Commitments

MVP uses cloud-assisted AI through OpenRouter, with OpenAI as the default configured provider. Privacy is a core feature, not a stretch goal.

Privacy controls:

- Read-Only GitHub App Permissions: We utilize fine-grained GitHub App permissions set strictly to Read-Only for repository contents and metadata. OnboardBuddy is structurally incapable of writing, pushing, or modifying your repositories.
- GitHub permission mirroring: users can only analyze repos they already have access to.
- Respect `.gitignore`: excluded files are never read.
- Custom ignored paths: users can exclude additional directories.
- Automated Secret Redaction: Best-effort scanning to detect and exclude common credential formats, `.env` files, and high-entropy strings before processing.
- Clear notice before cloud-assisted analysis: users are informed before any code context is sent to the configured OpenRouter provider.
- Compact evidence bundles: only extracted structure and selected snippets are sent to the LLM, not full source files.
- Generated outputs treated as drafts until reviewed: no auto-publishing.
- No selling of user data.

Important limitation: Even when secrets are excluded, code structure and selected snippets may reveal sensitive architecture. Local/Ollama mode is planned for a later phase to address enterprise privacy requirements.

1.5 Non-Trivial Features

Feature 1 — Role-Based Onboarding Package and AST Parser

OnboardBuddy imports a GitHub repository into an OnboardBuddy project, saves project metadata, downloads a temporary GitHub archive snapshot for analysis, parses TypeScript/TSX using the TypeScript Compiler API, and generates role-based onboarding packages from extracted evidence. The app is web-first: users work through the browser dashboard while backend workers perform repository analysis.

This feature's non-trivial tasks:

- Secure GitHub repository access and analysis snapshots.
- TypeScript evidence extraction through AST parsing.
- Codebase Onboarding Package generation.
- Role-based onboarding paths.
- Source-grounded AI explanation and review.

Package structure (without subheaders):

1. Start Here / Repository Overview
2. Entry Points and Why They Matter
3. Request / Event / Job Lifecycles
4. Critical 25% Learning Path
5. Role-based Onboarding Paths
6. Interactive Codebase Walkthroughs
7. Workflow Guides with source receipts
8. Data Schema / Source of Truth
9. Safety rails and Tooling
10. Architecture and Dependency References
11. Documentation Health

Why feature is non-trivial:

- Repository import must respect GitHub access, branch selection, ignored paths, and secret filtering before any analysis.
- AST parsing must produce stable evidence: files, symbols, imports/exports, classes/interfaces, functions, callbacks, entrypoints, side effects, schemas, tests, docs, and source receipts.
- The same repository snapshot must generate different learning paths for backend, frontend, DevOps, QA, and general developers without duplicating the whole analysis.
- Generated explanations are allowed only after structure is extracted; they must remain draft, reviewable, and tied to source receipts.

Feature 2 — Graph Visualizer, Architecture Map, and Module Dependency Graph

Provides practical graph views derived from the code evidence model. The graph UI is not a raw dump of every node. It shows slices that answer developer questions: what the app is made of, what depends on what, and where a selected onboarding workflow travels.

This feature's non-trivial tasks:

- Pull-based architecture and module dependency references.
- Architecture map: high-level app/service/module boundaries for the project.
- Role-specific dependency graph: dependency slices (file dependency) for the selected onboarding role.
- Class/interface graph: backend-oriented class, interface, inheritance, implementation, and service relationship view when available.
- Workflow graph: directed path for one selected workflow/tutorial.
- Source receipt drill-down: graph nodes link back to real files, symbols, and line ranges.

Why non-trivial:

- Large repository graphs become unreadable quickly; the product must cluster and filter rather than render everything.
- Architecture is a different abstraction level from dependency edges. The UI must separate "what this app is made of" from "which files/classes call or import each other."
- Dependency graphs must be role-based. A backend developer should see routes, services, repositories, schemas, jobs, and backend tests first; a frontend developer should see pages, components, state, API clients, and frontend tests first.

Feature 3 — Generic Workflow Extraction and Walkthrough/Tutorial Generation

Extracts cross-file workflows from the code evidence model, then generates walkthrough/tutorial experiences for the workflows that matter most. A workflow is discovered algorithmically from entrypoints to side effects (result data); the tutorial is the human-readable, step-by-step presentation of that workflow.

This feature's non-trivial tasks:

- Workflow extraction that is general about types of entrypoints (API endpoints, worker Redis queue enqueue, UI component fields)
- Interactive walkthrough is based on role-specific workflows (UI flow, API flow)

Supported target shapes include backend API, frontend app, full-stack app, browser extension, worker, queue processor, scheduled job, serverless app, library/package, and monorepo.

Why non-trivial:

- The extractor cannot rely only on typical API routes. It must detect routes, UI route/page entypoints, event listeners, job processors, scheduled handlers, serverless handlers, public package exports, and configured runtime entry files.
- It must trace both forward from entypoint to side effect and backward from side effect to entypoint.
- Raw call chains are too noisy for onboarding. The system must collapse implementation details into useful tutorial stops such as trigger, handler, service/domain logic, schema/API access, and output.
- Tutorial granularity is adaptive. For broad workflows, stops should be service/module-level; for confusing or critical logic, stops can drill down to specific functions, classes, or schema operations.
- Walkthroughs must be role-specific and generation-backed: users can open any role's walkthrough area, but steps appear only after that role package has been generated.

Feature 4 — Critical 25% / Critical Path Identification

Identify the Critical 25% of onboarding paths from extracted candidates. In this design, a **path** is a candidate execution trace from an entypoint through handlers/services/modules toward an output or side effect. It is not promoted to an onboarding workflow until it has been ranked and selected into the Critical 25%.

This feature's non-trivial tasks:

- Critical 25% of files, data, paths must be role-specific and useful for a developer/
- Path identification must find all possible paths from entry points and side effects and only then it should filter. It's better to have some unuseful info, rather than lose critical ones.
- Workflow granularity should be high-level enough for users to understand it, it should be a path through services and modules, and only rarely include functions.

Critical Path Score:

- 25% entypoint exposure
- 20% downstream impact
- 20% structural centrality
- 15% external side effects
- 10% documentation gap
- 5% test/coverage signal
- 5% git churn/recency

Why non-trivial:

- Churn alone is misleading; important code may be stable and noisy files may change often without teaching the repo.
- The ranker must explain its reasons so users trust why a candidate path becomes part of the first 25%.
- Role weighting must change the output without changing the underlying snapshot. Health checks may matter more for DevOps than backend, while API clients may matter more for frontends.

Feature 5 — Incremental Re-analysis

Update an existing project analysis from a new commit without re-scraping and regenerating everything from scratch. Incremental re-analysis compares snapshots, re-parses changed files, updates affected code evidence model records, re-runs impacted path/workflow ranking, and marks only affected generated onboarding docs as stale.

This feature's non-trivial tasks:

- Data stored from AST Parser must have enough data not only for generating sound explanations, but also for repeated regeneration.
- Re-analysis can be automatically triggered by GitHub push webhook or manually (default option), it has to show that the package is stale and needs re-analysis.

Why non-trivial:

- Re-running the entire parser/ranker/generator for every commit is expensive and noisy.
- It must detect changed files and symbols, update only affected model records, and preserve stable IDs where possible.
- It must connect changed files and symbols to selected workflows, package sections, and source receipts.
- It must preserve review history while marking only impacted generated sections stale.
- It must avoid pretending every nearby code change invalidates onboarding documentation.

1.6 Standard Features

- User authentication (Supabase Auth + GitHub OAuth connection):
 - Sign-up and login through Supabase Auth using GitHub OAuth. Supabase handles sessions, token refresh, and account state. The GitHub OAuth token is stored encrypted and used for repository access. Scope: Supabase client setup, OAuth callback route, session middleware on the backend, one users row and one github_connections row per account.
- Repository import + branch selection (GitHub App):
 - User selects, after connecting to GitHub App, a GitHub repository, picks a branch, and confirms the import. The backend validates repo access via the user's GitHub token and creates a project record. Scope: one form, one API endpoint, one validation call to the GitHub API, one projects row.
- Project dashboard navigation:
 - After login, the user sees a project list, add project, recent activity. Selecting a project opens the project view with tabs/navigation for Overview, Onboarding, Walkthrough, Graphs, and Settings. Scope: two route-level pages (project list, project detail) with a tab layout and conditional rendering per permission tier.
- Project CRUD (create, view, update settings, archive/delete):
 - Standard create/read/update/delete operations on projects. Update covers renaming, changing the analyzed branch, and editing ignored paths. Archive soft-deletes the project. Scope: REST endpoints for each operation, permission checks, and corresponding UI forms.
- Project team invitations:
 - Owner or Admin invites members by email with an assigned permission tier and an optional developer role. Invited users see pending invitations on their dashboard and can accept or decline. Invitations can be revoked by the inviter. Scope: invitation CRUD endpoints, email-matching on login, a pending invitations list page, and status transitions (pending -> accepted / revoked / expired).
- Three project permission tiers (Owner, Admin, Developer):

- Each project member has one tier. Owner has full control including project deletion and ownership transfer. Admin can manage members and settings but cannot delete the project. Developer can view content, choose their role, and request package generation but cannot overwrite existing packages or re-run analysis. Scope: a `permission_tier` column on `project_members`, middleware checks on every protected endpoint.
- Developer-role preference selector:
 - When joining a project or from settings, a member picks their default developer role (Backend / Frontend / DevOps / QA / General). This determines which onboarding package they see first. Scope: one dropdown, one API call to update `project_members.developer_role`.
- Settings (Privacy, AI provider, Project, Team):
 - Grouped settings views gated by permission tier. Privacy settings control ignored paths and privacy notice acceptance. AI provider settings configure the OpenRouter model. Project settings cover branch and repo metadata. Team settings show the member list and invitation management. Scope: one settings page with tabs, corresponding API endpoints for each group, permission-based visibility.
- Analysis run status:
 - After an analysis job is queued, the dashboard shows job progress (queued -> running -> complete / failed) with timestamps. Scope: blocking, parallel tasks and status endpoint that reads from the `analysis_jobs` table, a status badge component on the project overview.
- Package export as Markdown/zip:
 - Users can export a completed onboarding package as a Markdown file or a zip bundle containing all sections. Scope: one backend endpoint that assembles package sections into Markdown, a download button on the UI.
- Review/approval controls:
 - Owner or Admin can mark individual package sections as approved or request changes. Approved sections are visually distinguished. Scope: a `review_status` column on package sections, approve/reject buttons, an API endpoint to update status.
- Snapshot history (Stretch):
 - A list of past analysis snapshots for a project showing commit hash, branch, date, and trigger type. Users can view which snapshot the current package was generated from. Scope: a read-only list view, one API endpoint querying the `analysis_snapshots` table.

Permission rules:

- Owner: full project control, including deleting the project, inviting/kicking members, changing settings, approving package sections, and re-running analysis.
- Admin: can invite/kick Developers, change project settings, approve package sections, and re-run analysis.
- Developer: can view project content, choose their default developer role, open any role's onboarding/dependency graph, and request generation for any role package that does not exist yet. Developers cannot overwrite an existing role package and cannot re-run project analysis.

Visibility rules:

- Project Overview is visible once the project exists.
- Architecture Map is visible after the first successful analysis snapshot, is included in every role onboarding package, and describes the project-wide app/service architecture.

- Your Onboarding, Walkthrough, and role-specific Dependency Graph are openable for every role. If the selected role package does not exist, the page shows a missing state with a Generate button instead of hiding the view.
- Team and Project Settings are visible according to permission tier.

1.7 Risks

Risk	Problem	Mitigation
Privacy	Repos may contain proprietary code, secrets, internal architecture	Read-only OAuth, permission mirroring, ignored paths, secret filtering, no full repo storage, compact evidence bundles to LLM, clear privacy notices. Privacy is a core feature, not a stretch goal.
Cloud AI exposure	OpenRouter mode sends selected code context to a third-party provider, with OpenAI as the MVP default	Clear pre-analysis privacy notice, send only extracted evidence bundles (not full files). Local/Ollama or another provider can be configured later through OpenRouter.
Hallucination	AI may generate plausible but wrong explanations	Extraction-first pipeline: structure is deterministic, LLM only explains. Source receipts on every claim, confidence labels (High/Medium/Low), draft/review workflow, team approval before sharing.
Ranking accuracy	Churn may over-rank noisy files; stable critical code may be under-ranked	Low churn weight (5%), candidate-path-first ranking, composite scoring with 7 signals, explainable ranking reasons, role-based re-weighting.
Scalability	Large repos may be slow or produce noisy results	Configurable file count and LOC limits, progressive analysis (analyze by module/directory), chunked processing, incremental updates via snapshot diffs.
Scope	Full multi-language support is too large for MVP	TypeScript-first MVP with parser abstraction layer so new language parsers can plug in later.
Cost	Sending too much context to AI is expensive	Send compact evidence bundles only, cache AI outputs, regenerate only stale sections, track token usage per analysis.

Risk	Problem	Mitigation
Trust	Teams may not accept auto-generated docs	Draft-first generation, edit/approve workflow, source receipts, confidence labels, team review before sharing.

2. Feedback

Feedback 1 — Differentiation from Existing AI Tools

Source: TA, David Falade (peer), Daud Virk (peer)

Description: Multiple reviewers asked how OnboardBuddy is materially different from Claude Code, Cursor, or Windsurf, which can already generate code overviews and answer codebase questions. The TA specifically noted that removing the tutorial feature weakened differentiation, and that the remaining feature set could be described as "you burn tokens instead of us."

Action: The design emphasizes that OnboardBuddy is a documentation maintainer and its core value is its **deterministic analysis pipeline**: AST parsing, algorithmic workflow extraction, Critical 25% / critical path identification, and hash-based incremental re-analysis. The LLM is used only as an optional explanation layer on top of verified structure. Additionally, the **Interactive Codebase Walkthrough** feature has been re-introduced as a guided, step-by-step trace through selected workflows, derived from the code evidence model. Unlike AI chats, our onboarding is persistent, versioned, role-specific, and validated.

Feedback 2 — AI Hallucination and Accuracy

Source: Will Chen (peer), Ella McReynolds (peer), Daud Virk (peer)

Description: Reviewers expressed concern about AI-generated explanations being incorrect, which is particularly dangerous for onboarding materials that new developers would trust without the context to verify them.

Action: The design implements a multi-layer validation chain: (1) structure is extracted deterministically from AST/configs/schemas, no generation involved, (2) AI-generated sections include source receipts for the files, symbols, workflows, and schema references used as evidence, (3) confidence labels (High/Medium/Low) indicate whether the explanation is directly supported, inferred, or weakly supported, (4) all generated content starts as a draft and requires team review before being shared. The system distinguishes between deterministic outputs (graph, workflows, rankings) and AI-assisted outputs (explanations), making it clear what is provably grounded versus what is inferred.

Feedback 3 — Privacy and Security

Source: Ella McReynolds (peer), David Falade (peer), Linda Chu (peer)

Description: Several reviewers noted that privacy should not be a stretch goal, especially since the target users work with enterprise codebases. Concerns included leaking internal/confidential code and the risks of sending code to third-party AI services.

Action: Privacy has been elevated from a stretch goal to a **core feature**. The design includes: read-only OAuth (never writes to repos), GitHub permission mirroring, `.gitignore` respect, custom ignored paths, secret filtering (`.env`, keys, tokens, credentials), clear pre-analysis privacy notices, compact evidence bundles sent through OpenRouter to the configured provider (not full source files), no full repo storage by default, and no training/selling of user data. Local/Ollama or another provider can be configured later through the provider layer.

Feedback 4 — Scalability for Large Repositories

Source: Kreetik Thakur (peer), Daniel Lee (peer)

Description: Reviewers questioned how the system would handle very large repositories (millions of lines of code) without performance degradation or unusably noisy results.

Action: The design includes: configurable file count and LOC limits per analysis, progressive analysis (analyze by module or directory rather than entire repo at once), chunked AST processing, incremental updates via snapshot diffs (only re-analyze changed files), and graph pruning to filter noise. The system stores file and symbol hashes to enable efficient comparison across snapshots.

Feedback 5 — Multi-Language Support Path

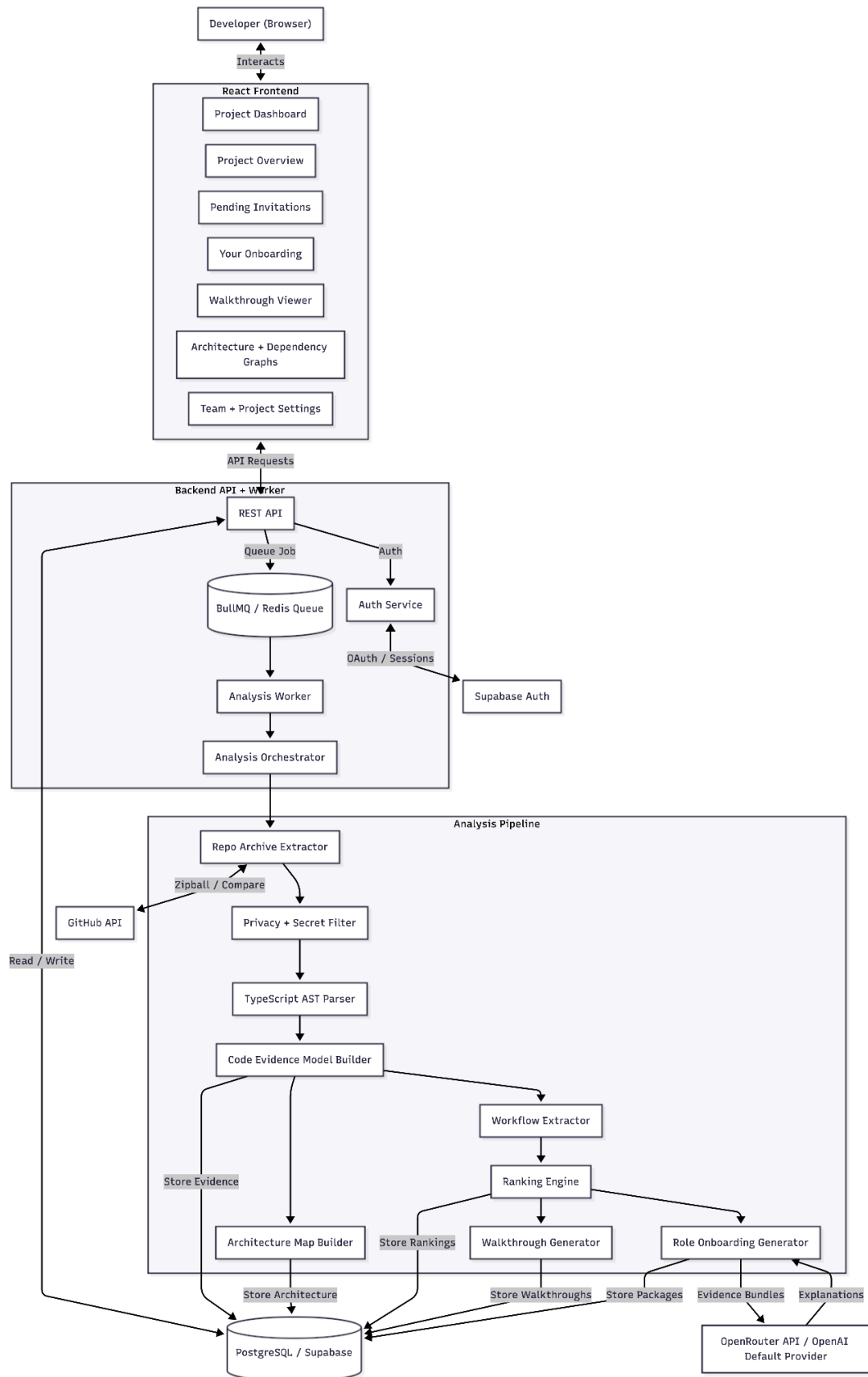
Source: Kreetik Thakur (peer)

Description: Reviewer asked how the system would eventually handle repositories with multiple languages or frameworks beyond the initial TypeScript support.

Action: The MVP is TypeScript-first, but the analysis pipeline is designed with a **parser adapter layer**. The code evidence model schema is language-agnostic (files, symbols, imports, calls, entrypoints, side effects). Only the AST extraction step is TypeScript-specific. Future language parsers can be added as internal adapters that produce the same model records and relationships, enabling multi-language support without redesigning the core pipeline.

3. Technical Design

3.1 Dataflow Diagram



3.2 Tech Stack and Architecture

Service Layers + Tech Stack

Layer	Technology	Purpose
Frontend	React + TypeScript + Tailwind CSS	Project dashboard, onboarding viewer, walkthrough viewer, workflow guides
Graph UI	React Flow / D3.js	Architecture maps, dependency graphs, workflow path visualization
Backend API	Node.js + Express + TypeScript	REST API, auth integration, project management, job enqueueing, typed error handling
Analysis Worker	Node.js + TypeScript	Repository archive extraction, analysis execution, workflow extraction, onboarding generation jobs
Static Analysis	TypeScript Compiler API	AST parsing, symbol extraction (fast, deterministic, thorough)
Queue	BullMQ backed by Redis	Manages the queue between API and worker
Auth	Supabase Auth with GitHub OAuth	Login, signup, session management
Database	Supabase PostgreSQL & MongoDB (tentative)	Projects, members, snapshots, code evidence model, workflows, onboarding packages, walkthroughs
Repo Access	GitHub OAuth + GitHub API + Zipball archives	OAuth, repo metadata, branch/commit lookup, archive download, compare/diffs
AI	OpenRouter with OpenAI as default provider	Semantic analysis + explanation generation from pre-extracted evidence bundles
Infra	Docker, GitHub Actions, Railway	Local dev environment, CI/CD, deployment

Main Services/Systems

Component	Responsibility
Web Dashboard	Repo import, project dashboard, graph rendering/layout, role onboarding viewer, walkthrough viewer, team tab, settings tab, review/export
Backend API	Auth, permissions, projects, settings, analysis job enqueueing, serving stored data to the frontend
Auth Service	GitHub OAuth flow, Supabase session validation, permission checks
Analysis Worker (+ Analysis Orchestrator)	Dequeues jobs, fetches/extracts GitHub archives, scans files, runs AST parsing, builds the code evidence model, extracts workflows
BullMQ / Redis	Queue backend for analysis and regeneration jobs
Repo Archive Extractor	Downloads temporary GitHub zipball, extracts files, discards archive after processing
Privacy + Secret Filter	Strips .env, API keys, tokens, credentials, and files matching .gitignore or custom ignored paths before any parsing
AST Parser	Extracts symbols, imports/exports, calls, classes, interfaces, entrypoints, side effects, and source positions using the TypeScript Compiler API
Code Evidence Model Builder	Assembles parsed AST output into the unified evidence model: files, symbols, relationships, hashes, and source receipts
Architecture Map Builder	Derives high-level app/service/module boundaries and clustering from the evidence model
Workflow Extractor	Traces candidate execution paths from entrypoints through calls, imports, callbacks, schemas, and side effects
Ranking Engine	Workflow scores, Critical 25%, role-based priorities
Walkthrough Generator	Step-by-step tutorial creation from ranked workflows and code evidence model
AI Generation Service (Role Onboarding Generator)	Evidence-backed explanation generation through OpenRouter, using OpenAI as the default provider
PostgreSQL (Supabase)	Projects, members, snapshots, evidence model entities, workflows, walkthroughs, onboarding packages, receipts

Pipeline Order and Rationale

1. Project/import metadata is saved before analysis so failed analysis still leaves a visible project record with errors.

2. Privacy filtering runs before AST parsing so ignored files and secret-like files never enter the parser, code evidence model, or AI evidence bundle.
3. AST parsing runs before code evidence model construction because the model should be built from structured TypeScript compiler facts rather than raw text.
4. Code evidence model construction happens immediately after parsing because it is the shared source for architecture map, workflow extraction, ranking, receipts, and incremental re-analysis.
5. Architecture map generation can happen before workflow extraction because it mostly needs project structure, imports, runtime config, and module/service clustering.
6. Workflow extraction runs from the code evidence model because entrypoints, calls, callbacks, imports, schemas, and side-effect candidates are easier to trace once normalized as relationships.
7. Critical 25% / critical path identification runs after candidate extraction because scoring depends on entrypoint exposure, downstream impact, centrality, side effects, docs, tests, and churn.
8. Onboarding packages and walkthroughs are generated last because they should present already-ranked evidence rather than invent structure.
9. Incremental re-analysis compares the new snapshot to the previous snapshot and updates only impacted model records, selected workflows, rankings, generated onboarding sections, and their saved generation context.

This order avoids a common implementation mistake: generating documentation directly from files first, then trying to recover structure later. In OnboardBuddy, documentation and visual graphs are views over the code evidence model.

Function Purpose and AI Evidence

For AI to “understand code”, the worker should save:

- Stable identity: file path, export path, qualified symbol name, symbol kind, parent symbol, and route/config registration when applicable.
- Shape: signature, parameters, return type, decorators, interface/class relationships, public/private visibility, and generic/type constraints.
- Behavior signals: calls made, callbacks registered, branches, thrown errors, async boundaries, database/schema access, HTTP/API calls, queue operations, file operations, auth checks, logging/monitoring calls, environment variables read, and external libraries used.
- Purpose signals: identifier names, nearby comments/JSDoc, string literals used as routes/events/messages, related tests, package/config context, and where the symbol sits in a workflow.
- Change signals: file hash, symbol/body hash, signature hash, and dependency edge hashes.

The AI explanation layer is then fed a bounded evidence bundle, not the whole repo blindly. A bundle includes selected snippets, source receipts, symbol metadata, upstream/downstream relationships, workflow position, role relevance, ranking reasons, and, during regeneration, the previous generated section plus the changed evidence. This should be enough for an AI assistant to explain “this class is important because it validates GitHub access before analysis jobs are created” rather than only “this class imports X and calls Y.”

Confidence depends on evidence quality. If purpose is supported by route names, types, tests, side effects, and workflow position, the explanation can be high confidence. If only names or comments suggest purpose, the section should be medium/low confidence and cite that limitation. The system should not claim runtime behavior that cannot be supported by parsed code, config, tests, or generated receipts.

Incremental Regeneration Logic

Incremental re-analysis must save enough information for regeneration, not just first-time generation. A one-line change inside a function is handled as follows:

1. The worker resolves the new commit, downloads the GitHub archive snapshot, and hashes files.
2. If a file hash changed, the worker re-parses that file and computes hashes for each stable symbol in it.
3. Stable symbol keys connect old and new snapshots using file path + qualified symbol/export identity, so a function can be matched even if its line numbers moved.
4. If a symbol/body hash changed, the worker checks whether that symbol appears in `workflow_steps`, `source_receipts`, role path generation context, dependency graph context, or package section generation context.
5. Only directly affected workflows, rankings, graph views, package sections, and walkthrough steps are marked stale or regenerated.
6. Regeneration receives the previous generated section, old receipts, new receipts, changed snippets, changed hashes, and stale reason. It can update the affected paragraph/step instead of rewriting the whole package.

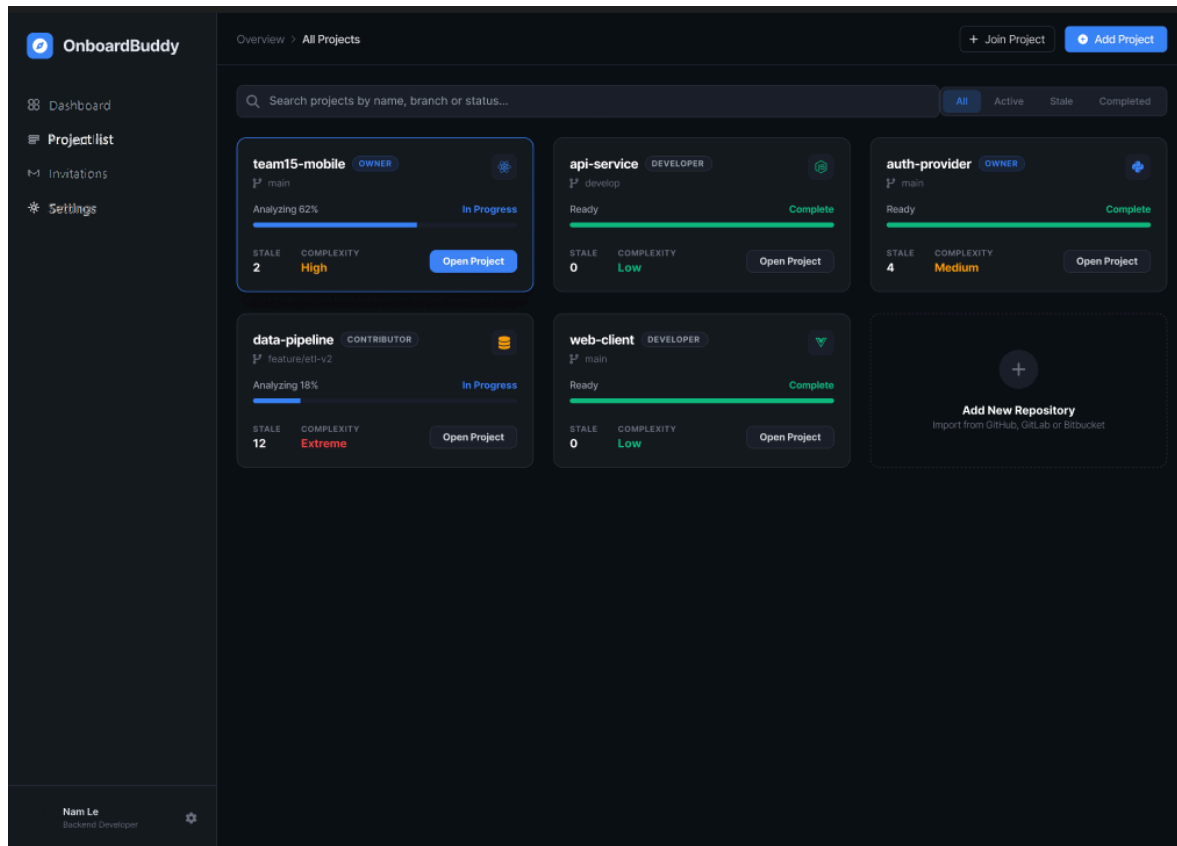
For example, if one line changes inside `AuthService`, the `AuthService` node hash changes. If the backend onboarding package cited that symbol in a source receipt or walkthrough step, that section is flagged stale with a reason like "Referenced function changed at commit abc123." If the changed function is not part of the Critical 25%, not in a generated package, and not connected to a selected workflow, the analysis snapshot updates but user-facing onboarding does not need regeneration.

AI Usage

OnboardBuddy uses OpenRouter with OpenAI as the default provider. The LLM is used as a semantic analysis and explanation layer. It receives a pre-extracted evidence bundle, which includes a file index, per-file symbol metadata (symbol names, kinds, signatures, call relationships, and imports), and a module dependency graph with nodes, edges, and detected entry points. Raw source file content is never sent. The LLM uses this structured output to generate human-readable explanations for onboarding package sections and walkthrough steps. Users are notified before any code context is sent.

3.3 UI Mockups

Screen 1 — Project List/Dashboard



Purpose: Landing page after login. Shows all projects the user can access. Difference between two tabs: dashboard shows recently ordered projects and recent activity, list shows all projects.

Layout:

- Top navigation bar with app logo, and left bottom panes is user avatar, and settings.
- Main content area with project cards.
- Each project card shows repo name, branch, analysis status, current stale count, and the user's permission tier.
- "Add Project" button opens the repository import flow.
- "Join Project" button opens pending project invitations for the user's email.

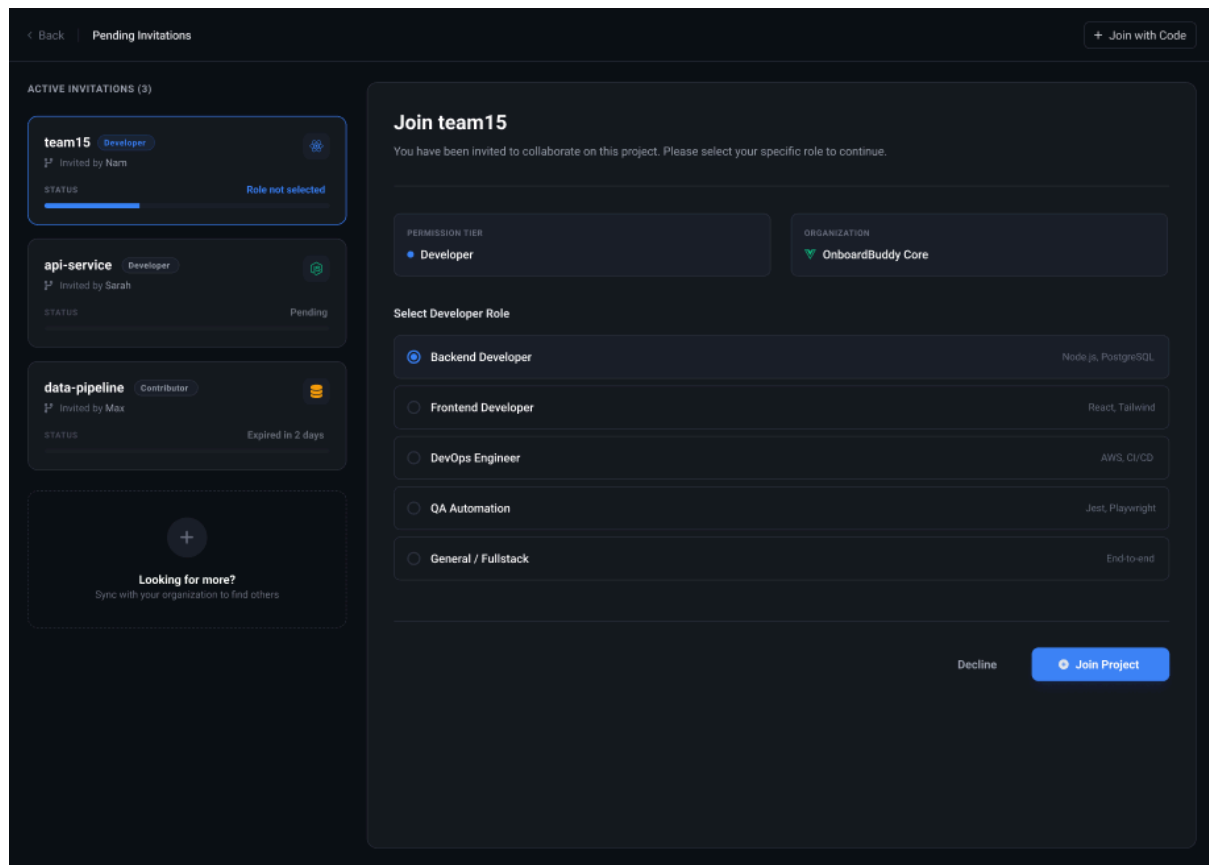
User Interactions:

- Click a project card -> navigates to that project's dashboard.
- Click "Add Project" -> navigates to Repository Import.
- Click "Join Project" -> navigates to Pending Invitations.
- Owner/Admin can archive or delete a project from card actions; Developer cannot.

API Endpoints Used:

Action	Method	Endpoint
Load user projects	GET	<code>/api/projects</code>
Load pending invitations	GET	<code>/api/invitations</code>
Delete/archive project	DELETE	<code>/api/projects/:id</code>

Screen 2 — Pending Invitations and Join Project



Purpose: Let a user accept project invitations sent to their email linked account in OnboardBuddy.

Layout:

- Pending invitations list showing project name, repo owner/name, invited permission tier, invited-by user, and whether a developer role was preselected.
- Selecting an invitation opens a short onboarding panel.
- Short onboarding panel shows the project name, permission tier, and developer-role selector only if the admin did not preassign a developer role.
- "Join Project" button accepts the invitation and creates the `project_members` row.

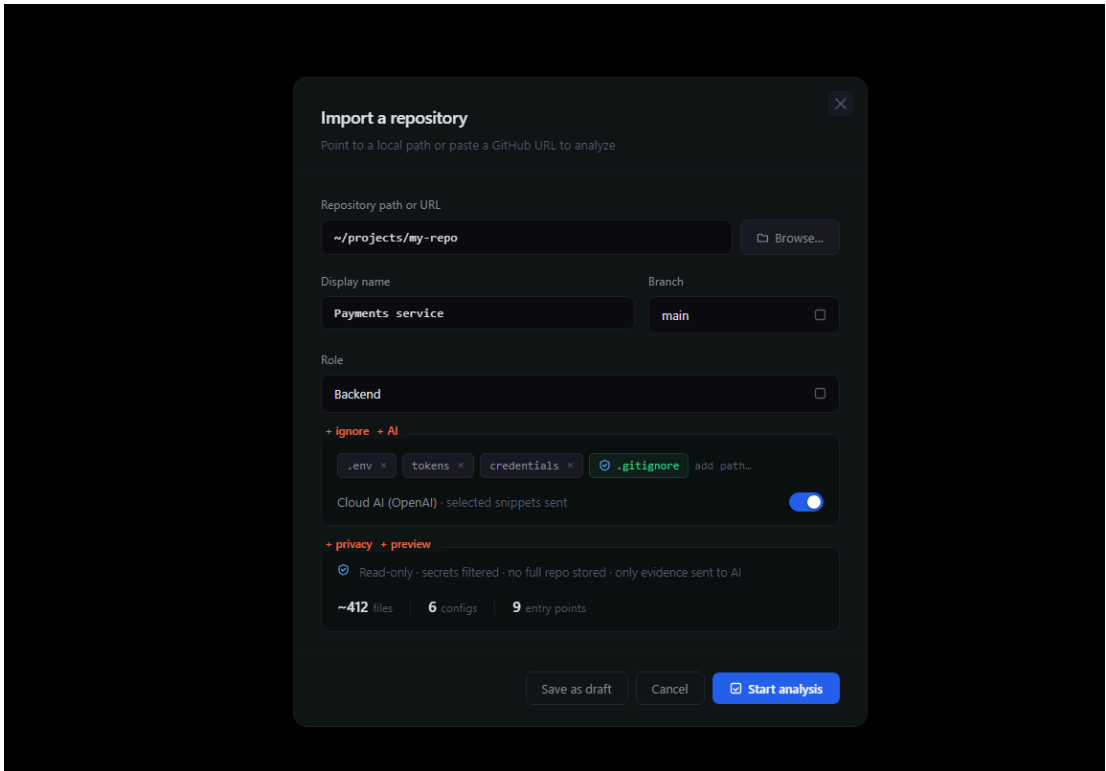
User Interactions:

- Click an invitation -> opens the short onboarding panel.
- If `developer_role` was provided by the inviter, the selector is shown as read-only or omitted.
- If `developer_role` is empty, user chooses Backend / Frontend / DevOps / QA / General.
- Click "Join Project" -> accepts invitation, adds user to project, and redirects to Project Overview.

API Endpoints Used:

Action	Method	Endpoint
Load pending invitations	GET	<code>/api/invitations</code>
Load invitation detail	GET	<code>/api/invitations/:invitationId</code>
Accept invitation	POST	<code>/api/invitations/:invitationId/accept</code>

Screen 3 — Repository Import and Privacy Setup



Purpose: Connect a GitHub repository, save it as an OnboardBuddy project, configure privacy settings, and start the first worker analysis job.

Layout:

- Step 1: GitHub connection status. If not connected, "Connect GitHub" triggers OAuth.
- Step 2: Repository selector populated from GitHub, then branch selector.
- Step 3: Initial developer-role selector: Backend / Frontend / DevOps / QA / General.
- Step 4: Privacy panel with ignored paths, secret filtering summary, and cloud-assisted AI toggle.
- Step 5: "Create Project and Analyze" button. Disabled until repo and branch are selected.

User Interactions:

- Select repository and branch -> creates a project record before analysis starts.
- Edit ignored paths -> updates project settings.
- Toggle cloud AI -> controls whether generated explanations use the configured OpenRouter provider; deterministic graph/ranking still run either way.
- Submit -> client calls the API to save the project, API creates `analysis_jobs` record, and the worker later fetches/parses the repo. The browser never receives full repository source.

API Endpoints Used:

Action	Method	Endpoint
Connect GitHub OAuth	GET	<code>/api/github/auth</code>
List user repos	GET	<code>/api/github/repos</code>
List repo branches	GET	<code>/api/github/repos/:owner/:repo/branches</code>
Create project	POST	<code>/api/projects</code>

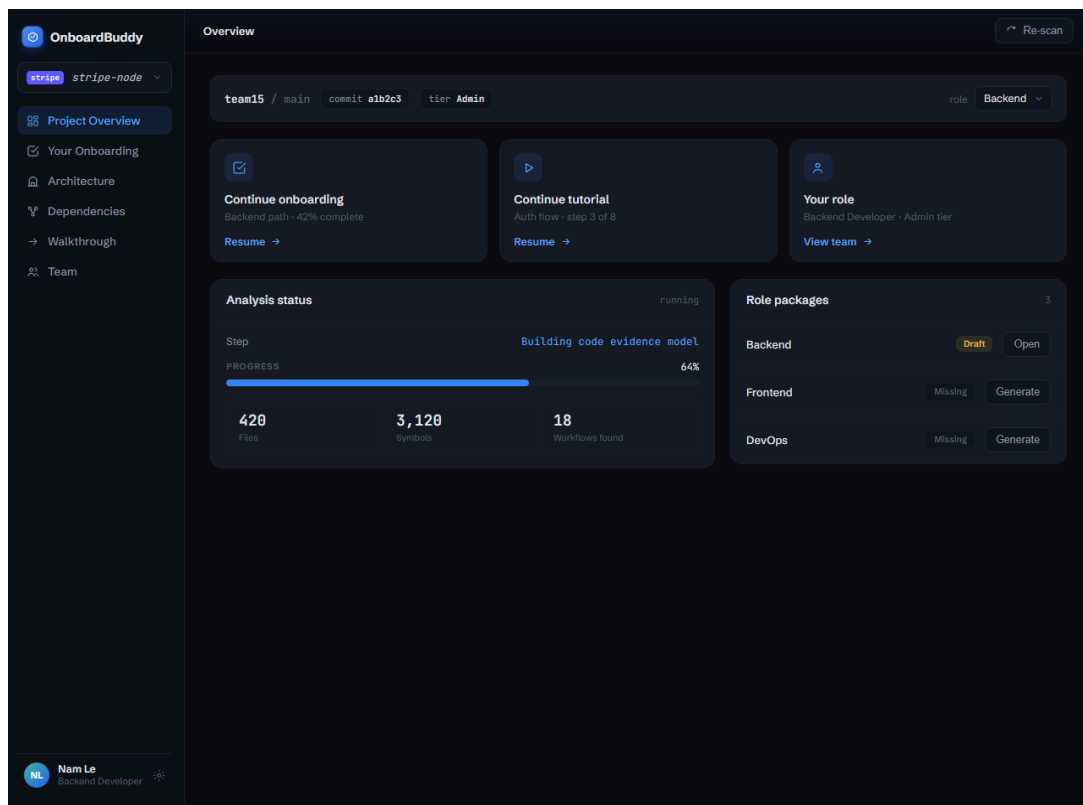
Action	Method	Endpoint
Update project settings	PUT	/api/projects/:id/settings
Start analysis	POST	/api/projects/:id/analyze

Screen 4 — Project Dashboard

Purpose: Main project shell after import. The dashboard is project-scoped and uses the following tabs:

- Project Overview
- Your Onboarding
- Architecture Map
- Dependencies
- Walkthrough
- Team
- Project Settings

Screen 4A — Project Overview



Purpose: Show project-level status before and after onboarding exists, quick-access launchpad for the project.

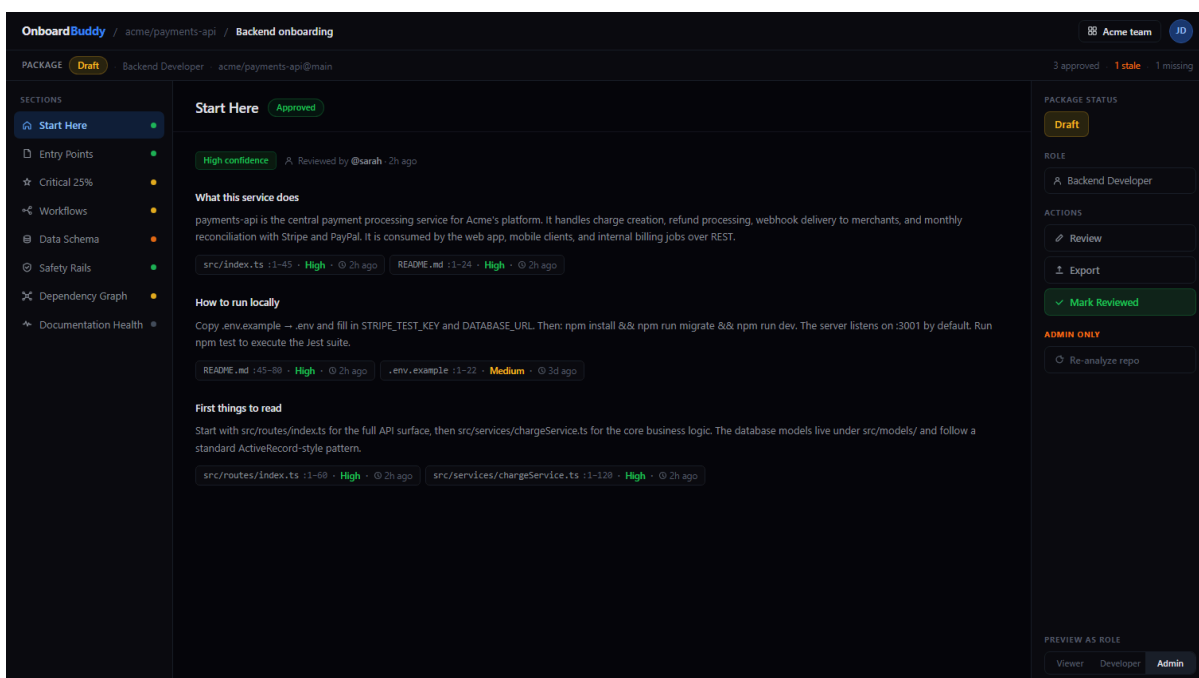
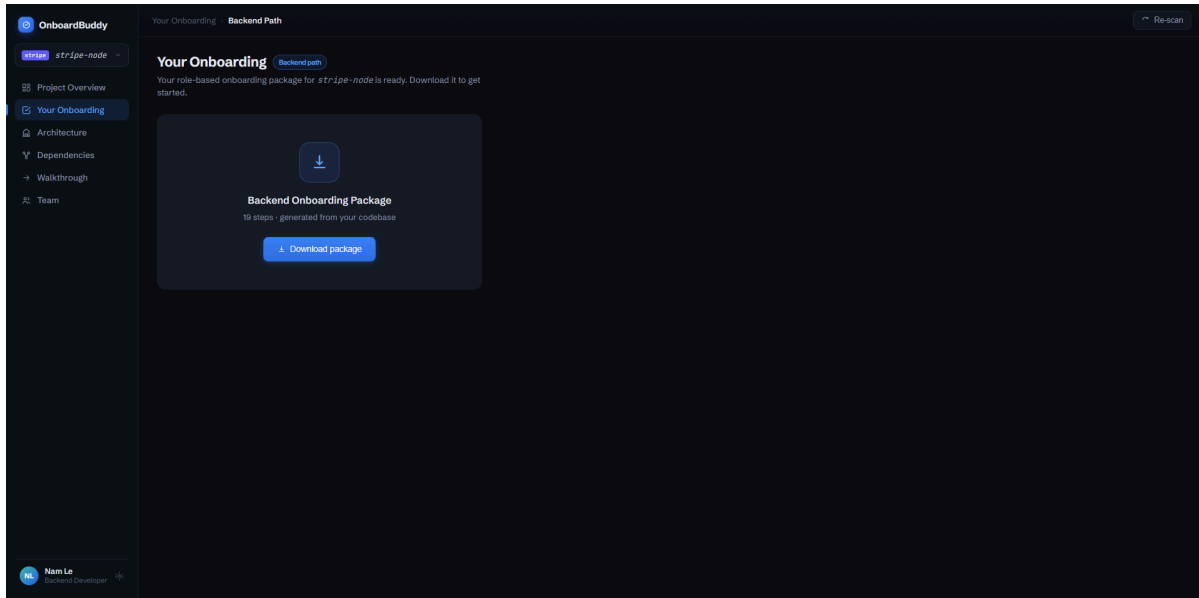
Layout:

- Analysis status card with current worker step and errors/warnings.
- Continue work buttons: onboarding, tutorial
- Repository facts: your role, branch, commit, file count, symbol count, workflow count.
- Onboarding availability by role: Backend, Frontend, DevOps, QA, General. Option to open or generate package.
- Stale-section summary after at least two snapshots.

User Interactions:

- Owner/Admin can start re-analysis.
- Any project member can open any role's onboarding package state.
- Any project member can request generation for a missing role package.
- If onboarding already exists for the selected role, the user sees the existing package and cannot recreate it unless they are Owner/Admin regenerating stale content.

Screen 4B — Your Onboarding



Purpose: Show the generated onboarding package or missing-package state for the selected developer role.

Layout:

- Package status: Missing / Generating / Draft / Approved / Stale.
- Section navigation: Start Here, Entry Points, Critical 25%, Workflows, Data Schema, Safety Rails, Dependency Graph, Documentation Health.

- Main section viewer with source receipts, confidence badges, stale indicators, and review status.
- Action area: Generate, Review, Export, Mark Reviewed. Re-analysis is shown only to Owner/Admin.
- Download package as a PDF, Markdown file.

User Interactions:

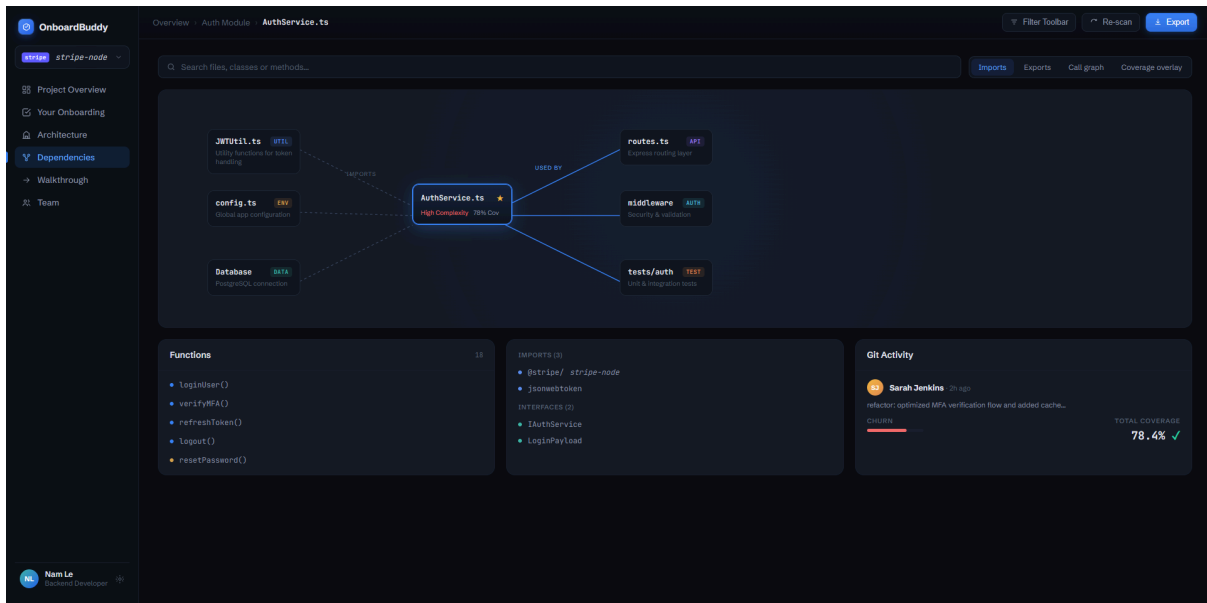
- Change developer role -> loads that role's onboarding package if it exists, or shows a missing state.
- Click Generate -> creates the role package if missing. Developers can request missing packages for any role, but cannot recreate existing packages.
- Click a source receipt -> opens code viewer at the referenced file/symbol/line range.
- Click a workflow -> opens the Walkthrough tab for that workflow.
- Owner/Admin can approve, edit, regenerate stale sections, and re-run analysis.

API Endpoints Used:

Action	Method	Endpoint
Load role package	GET	<code>/api/projects/:id/onboarding?role=backend</code>
Generate missing role package	POST	<code>/api/projects/:id/onboarding</code>
Load single section	GET	<code>/api/projects/:id/onboarding/sections/:sectionId</code>
Update review status	PATCH	<code>/api/projects/:id/onboarding/sections/:sectionId</code>
Regenerate stale section	POST	<code>/api/projects/:id/onboarding/sections/:sectionId/regenerate</code>
Export package	GET	<code>/api/projects/:id/onboarding/export?role=backend&format=markdown</code>

Screen 4C-D — Architecture Map + Dependency Graph





Purpose: Show the project-wide architecture map and role-specific dependency graphs.

Layout:

- Architecture Map tab: high-level app/service/module map generated once per snapshot and included in every role onboarding. Available after first successful analysis. Graph nodes: each box is a component (frontend module, service, gateway, database or worker); color shows its type (view only). Arrows: show how data moves: REST request, read/write queries, publish event, consume queue (view only).
- Dependency Graph tab: role-specific dependency slice. If onboarding does not exist for that role, shows a missing-package state and Generate button. Graph nodes: the centre file with its imports and used-by links (view only).
- Search bar: find service or component in system. Filter Toolbar: narrow graph by type or coverage.
- System / Data flow / Services / Coverage; Imports / Exports / Call graph / Coverage overlay tabs: switch graph view
- Side panel (below graph): node details, source receipts, connected workflows, importance score, and stale status.

User Interactions:

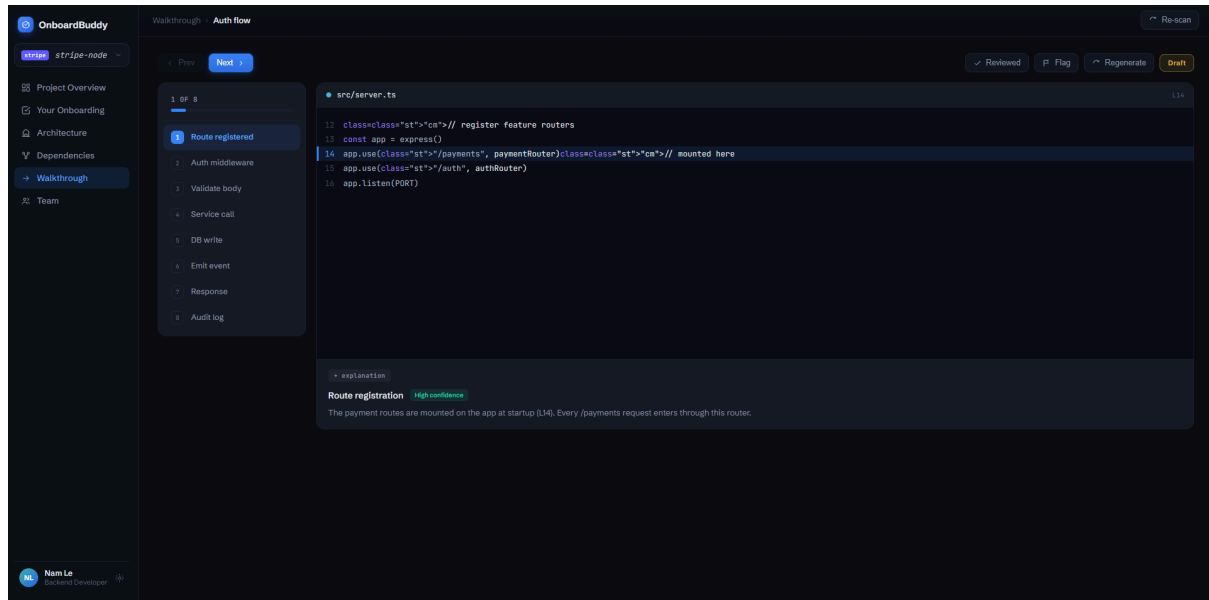
- Zoom and pan graph canvas. Read the information panel below to understand chosen node.
- Switch developer role -> dependency graph updates to that role's package, or shows a Generate action for the missing package.
- Select different view tabs to see different graphs.
- Search for modules via Search bar,

API Endpoints Used:

Action	Method	Endpoint
Load architecture map	GET	/api/projects/:id/graph/architecture
Load role dependency graph	GET	/api/projects/:id/graph/dependencies?role=backend
Load class/interface graph	GET	/api/projects/:id/graph/classes?role=backend
Load workflow graph	GET	/api/projects/:id/graph/workflows/:workflowId

Action	Method	Endpoint
Load node details	GET	/api/projects/:id/graph/nodes/:nodeId

Screen 4E — Walkthrough



Purpose: Guide developers step-by-step through a workflow/tutorial included in their role package.

Layout:

- Workflow picker filtered to workflows available for the selected developer role.
- Lower Header: Flag (flag for follow-up), Reviewed/Draft badge (whether the current step's explanation is still a draft or reviewed; turns Draft green), Regenerate button
- Main panel: code viewer with current stop highlighted.
- Explanation panel: what this stop does in the workflow context.
- Sidebar: Previous / Next controls, progress indicator, step list.

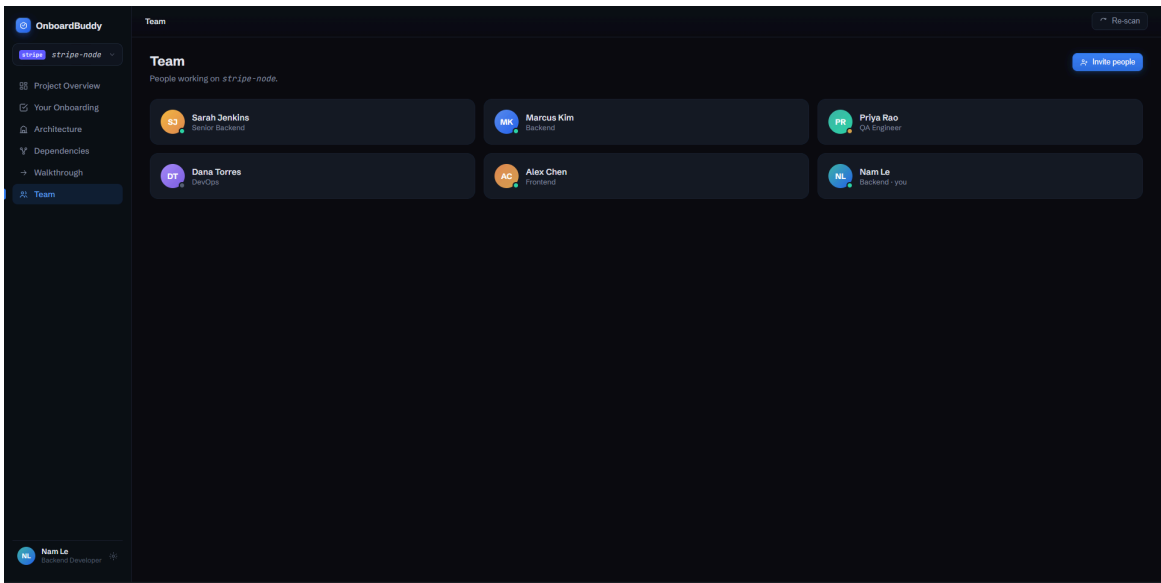
User Interactions:

- Select workflow -> loads tutorial steps.
- Click Previous / Next -> advances through stops.
- Click step or graph node -> jumps to that stop.
- Click source receipt -> opens referenced evidence.
- If selected role has no onboarding package, the tab shows a missing state explaining that walkthroughs are created from role onboarding and offers Generate.

API Endpoints Used:

Action	Method	Endpoint
Load workflows for role	GET	/api/projects/:id/workflows?role=backend
Load workflow detail	GET	/api/projects/:id/workflows/:workflowId
Load walkthrough steps	GET	/api/projects/:id/workflows/:workflowId/walkthrough?role=backend
Load file content for stop	GET	/api/projects/:id/files/:fileId

Screen 4F — Team



Purpose: Manage project membership and invitations.

Layout:

- Member table: name/email, selected developer role, status dot.
- Invite button for Owner/Admin. Invite modal and form.
- Member card for Owner/Admin: kick, change role/permission

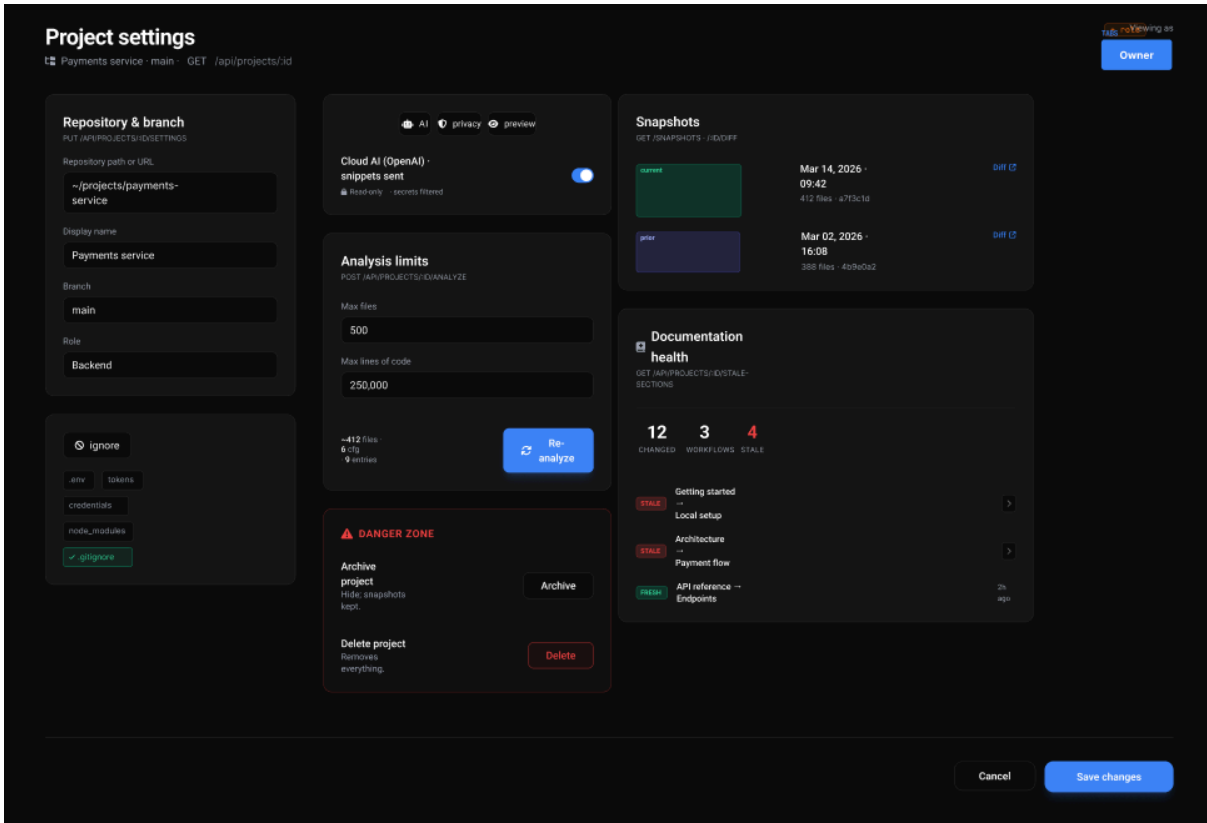
User Interactions:

- Owner/Admin can invite users by email and optionally preselect their developer role.
- Owner/Admin can revoke sent pending invitations.
- Owner/Admin can invite Developers and remove Developers.
- Owner can promote/demote Admins and transfer ownership.
- Developers can view members and change their own developer role, but cannot invite, kick, or change permissions.

API Endpoints Used:

Action	Method	Endpoint
Load project members	GET	<code>/api/projects/:id/members</code>
Invite member	POST	<code>/api/projects/:id/invitations</code>
Revoke invitation	PATCH	<code>/api/projects/:id/invitations/:invitationId</code>
Update member tier	PATCH	<code>/api/projects/:id/members/:userId</code>
Remove member	DELETE	<code>/api/projects/:id/members/:userId</code>

Screen 4G — Project Settings



Purpose: Configure project-level settings.

Layout: (one column, but for Design doc, moved into 3 column for viewability)

- Repository and branch metadata. Ignored paths.
- Cloud-assisted AI toggle and privacy notice. Analysis history and snapshot list.
- File/LOC analysis limits.
- Documentation Health panel with changed files, impacted workflows, and stale generated onboarding sections (Stretch).
- Danger zone for archive/delete.

User Interactions:

- Owner/Admin can update settings and start re-analysis.
- Developer can view settings but cannot update them.
- Owner/Admin can regenerate stale sections; Developer can only view stale status.

API Endpoints Used:

Action	Method	Endpoint
Load project details	GET	<code>/api/projects/:id</code>
Update settings	PUT	<code>/api/projects/:id/settings</code>
Start re-analysis	POST	<code>/api/projects/:id/analyze</code>
Load snapshots	GET	<code>/api/projects/:id/snapshots</code>
Load stale-section report	GET	<code>/api/projects/:id/stale-sections</code>
Load diff details	GET	<code>/api/projects/:id/snapshots/:snapshotId/diff</code>

3.4 RESTful API

All endpoints are prefixed with `/api`. Authenticated endpoints require a valid Supabase session token in the `Authorization` header. Responses use standard HTTP status codes. All request/response bodies are JSON unless otherwise noted.

API Error Behaviors

Status	Meaning	Example
400	Role parameter is missing or has invalid type	Invalid <code>role</code> value
401	User is not authenticated	Missing/expired Supabase session
403	User is authenticated but lacks project permission	Developer tries to re-run project analysis
404	Resource does not exist or is not visible to this user	Invitation ID not found for current user's email
409	Request conflicts with existing state	Role onboarding package already exists; duplicate pending invitation
422	Role value is validly provided but not allowed for this invitation	Accepting invitation without choosing a role when no role was preassigned

Permission checks must happen in the API layer for every project-scoped endpoint. The frontend may hide actions, but hidden buttons are not the security boundary.

Authentication

Method	Endpoint	Description	Request Body	Response
POST	<code>/api/auth/signup</code>	Register a new user	<code>{ email, password }</code>	<code>{ user: { id, email } }</code>
POST	<code>/api/auth/login</code>	Login	<code>{ email, password }</code>	<code>{ user: { id, email }, session: { access_token } }</code>
POST	<code>/api/auth/logout</code>	Logout	—	<code>{ success: true }</code>
GET	<code>/api/auth/me</code>	Get current user	—	<code>{ user: { id, email, created_at } }</code>

GitHub OAuth

Method	Endpoint	Description	Request Body	Response
GET	<code>/api/auth/github</code>	Initiate GitHub OAuth (Supabase handles login + GitHub token in one flow)	—	Redirect to GitHub
GET	<code>/api/auth/github/callback</code>	OAuth callback, Supabase session + GitHub token stored	—	Redirect to dashboard

GitHub Repo

GET	<code>/api/github/repos</code>	List user's accessible repos	—	<code>{ repos: [{ owner, name, private, default_branch }] }</code>
GET	<code>/api/github/repos/:owner/:repo/branches</code>	List branches for a repo	—	<code>{ branches: [{ name, commit_sha }] }</code>

Projects

Method	Endpoint	Description	Request Body	Response
GET	<code>/api/projects</code>	List user's projects	—	<code>{ projects: [{ id, repo_name, repo_owner, branch, status, permission_tier, last_analyzed_at, stale_count }] }</code>
POST	<code>/api/projects</code>	Create a saved project from a GitHub repo	<code>{ repo_owner, repo_name, branch, default_develop, per_role }</code>	<code>{ project: { id, repo_name, repo_owner, branch, status, permission_tier } }</code>
GET	<code>/api/projects/:id</code>	Get project details	—	<code>{ project: { id, repo_name, repo_owner, branch, status, </code>

Method	Endpoint	Description	Request Body	Response
				settings, permission_tier, onboarding_roles, last_snapshot } }
PUT	/api/projects/ :id/settings	Update project settings; Owner/Admin only	{ ignored_paths, ai_enabled, default_developer_role, file_limit?, loc_limit? }	{ settings: { ... } }
DELETE	/api/projects/ :id	Delete/archive a project	—	{ success: true }
GET	/api/projects/ :id/snapshots	List analysis snapshots	—	{ snapshots: [{ id, commit_hash, created_at, status }] }

Invitations

Method	Endpoint	Description	Request Body	Response
GET	/api/invitations	List pending invitations for current user's email	—	{ invitations: [{ id, project_id, repo_name, repo_owner, permission_tier, developer_role, invited_by, status }] }
GET	/api/invitations/ :invitationId	Load invitation detail for short onboarding panel	—	{ invitation: { id, project, permission_tier, developer_role, invited_by } }
POST	/api/invitations/ :invitationId/accept	Accept invitation and join project	{ developer_role? }	{ project: { id, repo_name, repo_owner }, member: {

Method	Endpoint	Description	Request Body	Response
				<code>permission_tier, developer_role } }</code>

Analysis

Method	Endpoint	Description	Request Body	Response
POST	<code>/api/projects/:id/analyze</code>	Start initial analysis or incremental re-analysis; Owner/Admin only after initial import	<code>{ branch?, commit?, mode?: "initial" "incremental" }</code>	<code>{ analysis: { id, status: "queued", mode } }</code>
GET	<code>/api/projects/:id/analyze/status</code>	Get current analysis status	—	<code>{ status, progress_pct, current_step, errors }</code>
GET	<code>/api/projects/:id/snapshots/:snapshotId</code>	Get snapshot details	—	<code>{ snapshot: { id, commit_hash, file_count, symbol_count, workflow_count, created_at } }</code>
GET	<code>/api/projects/:id/snapshots/:snapshotId/diff</code>	Get incremental diff between snapshots	—	<code>{ changed_files, changed_symbols, impacted_workflows, stale_sections }</code>

Code Evidence Model

Method	Endpoint	Description	Request Body	Response
GET	<code>/api/projects/:id/graph/architecture</code>	Get project-wide architecture map	—	<code>{ nodes: [{ id, label, type, file_count }], edges: [{ source, target, type }] }</code>

Method	Endpoint	Description	Request Body	Response
GET	/api/projects/:id/graph/dependencies	Get role-specific dependency graph or missing-package state	?role=backend	{ role, status, can_generate, clusters?: [{ id, name, nodes, internal_edges }], cross_edges?: [...] } }
GET	/api/projects/:id/graph/classes	Get role-specific class/interface graph or missing-package state	?role=backend	{ role, status, can_generate, nodes?: [{ id, label, kind, file }], edges?: [{ source, target, type }]} }
GET	/api/projects/:id/graph/workflows/:workflowId	Get workflow path graph	?role=backend	{ nodes: [{ id, label, file, symbol }], edges: [{ source, target }]} }
GET	/api/projects/:id/graph/nodes/:nodeId	Get node details	—	{ node: { id, label, type, file, symbols, connected_workflows, composite_score } }

Workflows

Method	Endpoint	Description	Request Body	Response
GET	/api/projects/:id/workflows	List extracted workflows, optionally filtered to role package	?role=backend	{ workflows: [{ id, title, trigger_type, purpose, composite_score, step_count }]} }

Method	Endpoint	Description	Request Body	Response
GET	/api/projects/:id/workflows/:workflowId	Get workflow details	—	{ workflow: { id, title, trigger, purpose, files, schemas, composite_score, confidence } }
GET	/api/projects/:id/workflows/:workflowId/walkthrough	Get role tutorial/walkthrough steps	?role=backend	{ steps: [{ order, file, symbol, line_start, line_end, explanation, receipts }] }
GET	/api/projects/:id/files/:fileId	Get file content (snippet) for walkthrough	—	{ file: { path, content, language } }

Onboarding

Method	Endpoint	Description	Request Body	Response
GET	/api/projects/:id/onboarding	Get role onboarding package	?role=backend	{ onboarding: { status, can_generate, sections: [{ id, title, type, content, confidence, review_status, is_stale }] } }
POST	/api/projects/:id/onboarding	Generate missing role package; any project member allowed if package does not exist	{ role }	{ onboarding: { role, status: "queued" } }
GET	/api/projects/:id/onboarding	Get single section detail	—	{ section: { id, title, type, content,

Method	Endpoint	Description	Request Body	Response
	<code>/sections/:sectionId</code>			<code>confidence, receipts, review_status, is_stale} }</code>
PATCH	<code>/api/projects/:id/onboarding/sections/:sectionId</code>	Update section review status/content; Owner/Admin for approval	<code>{ review_status, edited_content ? }</code>	<code>{ section: { id, review_status } }</code>
POST	<code>/api/projects/:id/onboarding/sections/:sectionId/regenerate</code>	Regenerate a stale section; Owner/Admin only	—	<code>{ section: { id, status: "regenerating" } }</code>
GET	<code>/api/projects/:id/onboarding/export</code>	Export role package	<code>?role=backend&format=markdown zip</code>	File download
GET	<code>/api/projects/:id/onboarding/role-paths</code>	Get role-based learning paths	—	<code>{ paths: { backend: [...], frontend: [...], devops: [...], qa: [...], general: [...] } }</code>

Documentation Health

Method	Endpoint	Description	Request Body	Response
GET	<code>/api/projects/:id/stale-sections</code>	Get generated onboarding stale-section report	—	<code>{ changed_files, impacted_workflows, stale_sections }</code>
GET	<code>/api/projects/:id/docs</code>	Stretch: list linked repository docs	—	<code>{ docs: [{ id, path, linked_workflows, linked_symbols, is_stale }] }</code>
GET	<code>/api/projects/:id/docs/:docId</code>	Stretch: get linked doc details	—	<code>{ doc: { id, path, headings, linked_entities,</code>

Method	Endpoint	Description	Request Body	Response
				<code>last_hash, is_stale, stale_reason } }</code>

Project Members and Invitations

Method	Endpoint	Description	Request Body	Response
GET	<code>/api/projects/:id/members</code>	List project members	—	<code>{ members: [{ user_id, email, permission_tier, developer_role, joined_at }] }</code>
POST	<code>/api/projects/:id/invitations</code>	Invite by email; Owner/Admin only	<code>{ email, permission_tier, developer_role? }</code>	<code>{ invitation: { id, email, permission_tier, developer_role, status } }</code>
PATCH	<code>/api/projects/:id/invitations/:invitationId</code>	Revoke pending invitation; Owner/Admin only	<code>{ status: "revoked" }</code>	<code>{ invitation: { id, status } }</code>
PATCH	<code>/api/projects/:id/members/:userId</code>	Update permission tier or developer role	<code>{ permission_tier?, developer_role? }</code>	<code>{ member: { user_id, permission_tier, developer_role } }</code>
DELETE	<code>/api/projects/:id/members/:userId</code>	Remove member; Owner/Admin only	—	<code>{ success: true }</code>

3.5 Backend

Backend Process Split

The backend has two separate Node.js TypeScript processes:

- API server: handles Supabase session validation, GitHub OAuth callbacks, project CRUD, settings, project membership/invitations, permission checks, read endpoints, and job creation.
- Worker: claims queued `analysis_jobs`, downloads and extracts a temporary GitHub archive snapshot, applies privacy filters, runs AST parsing, builds evidence model data, extracts workflows, ranks Critical 25%, generates role onboarding packages, updates progress, and records failures.

The API never performs long-running repository analysis inside a request/response cycle. It writes durable job state to PostgreSQL and may also enqueue through BullMQ/Redis for worker coordination.

Database Schema

The application uses PostgreSQL via Supabase. The following describes the key tables, their purpose, and relationships.

Users and Auth

`users`

Column	Type	Description
id	UUID (PK)	Supabase Auth user ID
email	VARCHAR	User email
created_at	TIMESTAMP	Account creation time

`github_connections`

Column	Type	Description
id	UUID (PK)	Connection ID
user_id	UUID (FK → users)	Owning user
github_user_id	INTEGER	GitHub user ID
github_username	VARCHAR	GitHub username
access_token_encrypted	TEXT	Encrypted OAuth token
scopes	TEXT[]	Granted OAuth scopes
created_at	TIMESTAMP	Connection time

Project Membership

Membership is project-scoped for the MVP, and the third permission tier is `developer`.

`project_members`

Column	Type	Description
project_id	UUID (FK → projects)	Project
user_id	UUID (FK → users)	Member
permission_tier	VARCHAR	owner / admin / developer
developer_role	VARCHAR	backend / frontend / devops / qa / general
joined_at	TIMESTAMP	Join time

project_invitations

Column	Type	Description
id	UUID (PK)	Invitation ID
project_id	UUID (FK → projects)	Project
email	VARCHAR	Invited email
permission_tier	VARCHAR	owner / admin / developer
developer_role	VARCHAR, nullable	Optional initial developer role. If null, invitee chooses during join.
invited_by	UUID (FK → users)	Inviting Owner/Admin
accepted_by	UUID (FK → users, nullable)	User who accepted the invitation
status	VARCHAR	pending / accepted / revoked / expired
created_at	TIMESTAMP	Invitation creation time
expires_at	TIMESTAMP	Optional expiration time
accepted_at	TIMESTAMP	Acceptance time

Constraint: there should be a unique pending invitation per project and normalized email. This prevents duplicate pending invites for the same user while still preserving accepted/revoked invitation history.

Projects

projects

Column	Type	Description
id	UUID (PK)	Project ID
user_id	UUID (FK → users)	Owner
repo_owner	VARCHAR	GitHub repo owner
repo_name	VARCHAR	GitHub repo name

Column	Type	Description
branch	VARCHAR	Analyzed branch
status	VARCHAR	idle / analyzing / complete / failed
created_at	TIMESTAMP	Creation time
last_analyzed_at	TIMESTAMP	Last analysis time

project_settings

Column	Type	Description
project_id	UUID (FK → projects, PK)	Project
ignored_paths	TEXT[]	Paths to exclude from analysis
ai_enabled	BOOLEAN	Whether to use the configured OpenRouter provider for explanations
default_developer_role	VARCHAR	Default role lens for viewing
file_limit	INTEGER	Max files to analyze
loc_limit	INTEGER	Max lines of code to analyze

Analysis

analysis_snapshots

Column	Type	Description
id	UUID (PK)	Snapshot ID
project_id	UUID (FK → projects)	Project
commit_hash	VARCHAR	Analyzed commit
branch	VARCHAR	Analyzed branch
file_count	INTEGER	Number of files analyzed
symbol_count	INTEGER	Number of symbols extracted
workflow_count	INTEGER	Number of workflows found
status	VARCHAR	pending / running / complete / failed
warnings	JSONB	Analysis warnings
created_at	TIMESTAMP	Snapshot time

analysis_jobs

The API creates analysis/generation jobs and the worker claims them. BullMQ/Redis is used as the queue/cache, but PostgreSQL is the durable source of truth for status shown in the dashboard.

Column	Type	Description
id	UUID (PK)	Job ID
project_id	UUID (FK → projects)	Project
snapshot_id	UUID (FK → analysis_snapshots, nullable)	Snapshot created/updated by the job
requested_by	UUID (FK → users)	User who requested the job
job_type	VARCHAR	analyze_project / generate_onboarding / regenerate_section
role	VARCHAR	backend / frontend / devops / qa / general, nullable for project analysis
status	VARCHAR	queued / running / complete / failed
progress_pct	INTEGER	0-100 dashboard progress
current_step	VARCHAR	Human-readable worker step
error_message	TEXT	Failure reason
created_at	TIMESTAMP	Queue time
started_at	TIMESTAMP	Worker start time
finished_at	TIMESTAMP	Completion/failure time

Code Evidence Model

The implementation stores the code evidence model in node/edge tables because imports, calls, inheritance, docs, tests, and workflow relationships are naturally relational. The product term is "code evidence model"; "architecture map", "dependency graph", "class graph", and "workflow graph" are frontend visualizations derived from this model.

The saved model must support three uses: first-time onboarding generation, AI explanation grounding, and incremental regeneration. Therefore it must store stable identities and hashes, not only display labels and line numbers.

Saved evidence contract:

Evidence	Saved as	Used for
File identity	<code>graph_nodes.type = "file", file_path</code> , file hash	Detect changed files between snapshots
Symbol identity	<code>graph_nodes.stable_key, name, file_path</code> ,	Match the same function/class/interface across commits even when lines move

Evidence	Saved as	Used for
Symbol shape	<code>graph_nodes.metadata.signature, params, returnType, visibility, exports</code>	Explain API surface and detect breaking changes
Symbol behavior	<code>graph_edges</code> plus <code>graph_nodes.metadata.behaviorSignals</code>	Explain what a symbol does and what it touches
Purpose evidence	<code>graph_nodes.metadata.purposeSignals</code>	Ground "why this class/function matters" explanations
Change evidence	<code>graph_nodes.hash, metadata.signatureHash, metadata.bodyHash</code> , edge metadata hashes	Decide whether a symbol, signature, or dependency relationship changed
Generation evidence	<code>package_sections.generation_context, source_receipts</code>	Regenerate only impacted sections and explain stale reasons

graph_nodes

Column	Type	Description
id	UUID (PK)	Node ID
snapshot_id	UUID (FK → analysis_snapshots)	Snapshot
stable_key	VARCHAR	Stable identity such as <code>src/auth/AuthService.ts#AuthService.validateToken</code>
type	VARCHAR	file / symbol / class / interface / function / entrypoint / schema / test / doc
name	VARCHAR	Node name
file_path	VARCHAR	Source file path
line_start	INTEGER	Start line
line_end	INTEGER	End line
hash	VARCHAR	Content/body hash for change detection
metadata	JSONB	Structured metadata: signature, params, return type, exports, behavior signals, purpose signals, signature/body hashes

graph_edges

Column	Type	Description
id	UUID (PK)	Edge ID
snapshot_id	UUID (FK → analysis_snapshots)	Snapshot
source_node_id	UUID (FK → graph_nodes)	Source node
target_node_id	UUID (FK → graph_nodes)	Target node
type	VARCHAR	imports / calls / extends / implements / registers_callback / touches_schema / references / tests / documents
metadata	JSONB	Additional edge metadata

Workflows

workflows

Column	Type	Description
id	UUID (PK)	Workflow ID
snapshot_id	UUID (FK → analysis_snapshots)	Snapshot
title	VARCHAR	Workflow title
trigger_type	VARCHAR	http_route / event / worker_job / scheduled / extension_event / build_deploy
purpose	VARCHAR	auth / request_lifecycle / data_read_write / config / deploy / logging / messaging / integration / monitoring
composite_score	DECIMAL	Composite importance score
confidence	VARCHAR	high / medium / low
metadata	JSONB	Additional workflow data

workflow_steps

Column	Type	Description
id	UUID (PK)	Step ID
workflow_id	UUID (FK → workflows)	Parent workflow
step_order	INTEGER	Position in sequence
node_id	UUID (FK → graph_nodes)	Graph node for this stop

Column	Type	Description
file_path	VARCHAR	File path
symbol_name	VARCHAR	Symbol name
line_start	INTEGER	Start line
line_end	INTEGER	End line
explanation	TEXT	AI-generated or template explanation
role_relevance	JSONB	<code>{ backend: 0.9, frontend: 0.2, ... }</code>

workflow_scores

Column	Type	Description
workflow_id	UUID (FK → workflows, PK)	Workflow
entrypoint_exposure	DECIMAL	25% weight
downstream_impact	DECIMAL	20% weight
structural_centrality	DECIMAL	20% weight
external_side_effects	DECIMAL	15% weight
documentation_gap	DECIMAL	10% weight
test_coverage_signal	DECIMAL	5% weight
git_churn_recency	DECIMAL	5% weight
ranking_reasons	TEXT[]	Human-readable reasons

Onboarding Package

onboarding_packages

Column	Type	Description
id	UUID (PK)	Package ID
snapshot_id	UUID (FK → analysis_snapshots)	Snapshot used for this package
project_id	UUID (FK → projects)	Project
role	VARCHAR	backend / frontend / devops / qa / general
status	VARCHAR	generating / draft / approved / stale / failed

Column	Type	Description
generated_by	UUID (FK → users)	User who requested generation
analyzed_commit	VARCHAR	Commit hash this package was generated from
created_at	TIMESTAMP	Generation time
updated_at	TIMESTAMP	Last update time

Constraint: unique (`project_id`, `role`, `analyzed_commit`) prevents duplicate packages for the same role and commit. The API returns `missing` as a computed state when no row exists for a requested role.

package_sections

Column	Type	Description
id	UUID (PK)	Section ID
package_id	UUID (FK → onboarding_packages)	Parent role package
snapshot_id	UUID (FK → analysis_snapshots)	Snapshot
type	VARCHAR	start_here / entry_points / critical_25 / workflow_guide / data_schema / safety_rails / architecture / dependency_graph / doc_health
title	VARCHAR	Section title
content	TEXT	Generated content (Markdown)
confidence	VARCHAR	high / medium / low
review_status	VARCHAR	draft / approved / edited / stale / regenerate_requested
generation_context	JSONB	Saved input context: workflow IDs, node stable keys, receipt IDs, generator/template version, evidence hashes
created_at	TIMESTAMP	Generation time
reviewed_at	TIMESTAMP	Last review time
reviewed_by	UUID (FK → users, nullable)	Reviewer

source_receipts

Column	Type	Description
id	UUID (PK)	Receipt ID

Column	Type	Description
section_id	UUID (FK → package_sections)	Linked section
node_id	UUID (FK → graph_nodes, nullable)	Linked graph node
node_stable_key	VARCHAR	Stable symbol/file key used to match against later snapshots
node_hash	VARCHAR	Hash observed when this receipt was generated
file_path	VARCHAR	Referenced file
symbol_name	VARCHAR	Referenced symbol
snippet	TEXT	Cited code snippet
commit_hash	VARCHAR	Commit where this was observed

Documentation Health

MVP documentation health applies to generated OnboardBuddy onboarding sections. [repo_doc](#) linkage is kept in the schema as a stretch-ready target for future README/docs/ADR/JSDoc analysis, but arbitrary existing repo docs are not required for MVP stale detection.

doc_links

Column	Type	Description
id	UUID (PK)	Link ID
snapshot_id	UUID (FK → analysis_snapshots)	Snapshot
doc_path	VARCHAR	Path to the documentation file
linked_node_id	UUID (FK → graph_nodes, nullable)	Linked code entity
linked_workflow_id	UUID (FK → workflows, nullable)	Linked workflow
doc_hash	VARCHAR	Hash of the doc file at analysis time

stale_flags

Column	Type	Description
id	UUID (PK)	Flag ID
snapshot_id	UUID (FK → analysis_snapshots)	Current snapshot
target_type	VARCHAR	package_section / repo_doc
target_id	UUID	ID of the stale section or doc link

Column	Type	Description
reason	TEXT	Why it was flagged (e.g., "AuthService changed")
changed_files	TEXT[]	Files that triggered the flag
created_at	TIMESTAMP	Flag time

Role Paths

role_paths

Column	Type	Description
id	UUID (PK)	Path ID
package_id	UUID (FK → onboarding_packages)	Role package
step_order	INTEGER	Order in the learning path
workflow_id	UUID (FK → workflows, nullable)	Linked workflow
section_id	UUID (FK → package_sections, nullable)	Linked package section
title	VARCHAR	Step title
reason	TEXT	Why this step matters for this role

Constraint: Constraint: at least one of `workflow_id` or `section_id` must be non-null.

Tentative: Snapshot Storage (MongoDB)

The raw analysis snapshot (snap.json) is stored as a document in MongoDB. Each document contains the full output from the AST parser: the file index, per-file symbol analyses, and the module dependency graph.

MongoDB is used here because the snapshot is a nested document retrieved as a whole, not queried field-by-field. PostgreSQL handles all relational data. The snapshot document links back to PostgreSQL via `snapshot_id`.

Third-Party Services

Service	Usage	Data Sent
GitHub API	OAuth authentication, repo listing, branch listing, commit lookup, compare API, zipball/tarball archive download	OAuth tokens, repo metadata requests
OpenRouter API	Generate human-readable explanations for package sections and walkthrough steps, with OpenAI as the default provider	Compact evidence bundles: workflow paths, involved files/symbols, selected snippets, schema references, config references, test references, commit hash. Never full source files.

Service	Usage	Data Sent
Supabase	Authentication, PostgreSQL database hosting	User credentials (handled by Supabase Auth), all application data
MongoDB	Store generated snapshots	Worker generated snapshot
BullMQ / Redis	Queue analysis and regeneration jobs for the backend worker	Job IDs, project IDs, snapshot IDs, status metadata

Data Access Patterns

- The frontend accesses application data through the RESTful API, no direct database access from the frontend.
- The backend API and worker use parameterized queries via a PostgreSQL client library to prevent SQL injection.
- Long-running analysis work is queued in BullMQ/Redis and executed by the backend worker, separate from the API process.
- Visualization queries (architecture, dependencies) use recursive CTEs or adjacency list traversals.
- Workflow step queries join `workflow_steps` with `graph_nodes` to resolve file paths and symbols.
- Incremental re-analysis compares file and symbol hashes between consecutive snapshots, then updates affected model records and generates onboarding stale flags.
- Raw analysis snapshots (snap.json) are written to MongoDB by the worker after each analysis run and retrieved as whole documents by `snapshot_id`.

4. Test Coverage/Plans

Frontend Tests

Framework: Vitest + React Testing Library for component tests, Playwright for E2E tests.

Test Area	Test Cases
Authentication	Login form validation, successful login redirect, logout clears session, protected route redirect
Dashboard	Project list renders, empty state shown, project card displays status/tier/stale count, delete project confirmation is Owner/Admin gated
Invitations	Join Project opens pending invitations, invitation detail opens short onboarding panel, role selector appears only when role was not preassigned, accept redirects to Project Overview
Repository Import	GitHub connection flow, repo dropdown populates, branch dropdown populates, privacy notice displayed, ignored paths editable, analysis starts on submit
Project Overview	Analysis status renders, role availability renders, Developer sees generate-once state, Owner/Admin sees re-analysis action
Your Onboarding	Sections render correctly, role selector changes content, source receipts are clickable, confidence badges display, stale badges display, export triggers download
Walkthrough Viewer	Steps render in order for an existing role package, missing-role generation state renders, navigation works, code viewer highlights correct lines, progress indicator updates
Architecture Map	Project-wide architecture renders after analysis, role dependency graph shows generated graph or missing-role generation state, class/interface graph tab renders when available
Documentation Health	Stale sections listed, regenerate button is Owner/Admin gated, mark reviewed clears stale flag, diff view opens
Review Controls	Draft/Approved/Stale status displayed, review modal opens, approval updates status
Team	Member list renders, invitations are Owner/Admin gated, Developer cannot kick/invite/change settings

E2E Test Scenarios:

1. Full onboarding flow: signup → connect GitHub → import repo → save project → run analysis → generate role onboarding → navigate walkthrough → export.
2. Role switching: change developer role → verify generated package content or missing-package generation state, learning path, dependency graph, and walkthrough state change.

3. Incremental re-analysis: Owner/Admin re-analyzes after code change → stale badges appear → stale section regenerated/reviewed → stale cleared.
4. Permission flow: Owner invites Developer → Developer opens another role and generates missing role onboarding once → Developer cannot recreate, re-analyze, kick members, or edit settings.
5. Invitation flow: Owner/Admin invites by email → invited user clicks Join Project → selects pending invitation → chooses developer role if needed → joins project.

Backend API Tests

Framework: Mocha + Chai + Supertest for HTTP integration tests.

Endpoint Group	Test Cases
Auth	Signup with valid/invalid data, login with correct/incorrect credentials, logout invalidates session, <code>/me</code> returns current user, other projects are hidden to invalid users
GitHub	OAuth redirect URL correct, callback stores token, repos list returns user's repos, branches list returns correct branches, unauthorized access blocked, handling error codes
Projects	Create with valid data, create with missing fields rejected, list returns only accessible projects, get returns project detail, delete/archive is permission gated, settings update persists
Invitations	Invite by email creates pending invitation, current user lists invitations by email, accept creates project member, preassigned developer role cannot be overridden unintentionally, revoked invitations cannot be accepted
Analysis	Start analysis returns queued status, Owner/Admin incremental re-analysis is allowed, Developer re-analysis is blocked, status endpoint returns progress, snapshot created on completion, diff between snapshots returns correct changed files/symbols, check draft → approved transition, concurrent analyses from multiple users
Code Evidence Model	Architecture endpoint returns nodes/edges, role dependency endpoint returns generated graph or missing-package state, class graph returns relationships or missing-package state, workflow graph returns path, node detail returns connected info
Workflows	List returns extracted workflows, detail returns full workflow data, walkthrough returns ordered steps with receipts
Onboarding	Missing role package can be generated once for any role, existing role package cannot be recreated by Developer, role filter changes section emphasis, section update changes review status, export produces valid Markdown/zip
Incremental Re-analysis	Re-analysis updates changed files/symbols, preserves unaffected model records where possible, marks impacted generated sections stale, Owner/Admin regeneration clears stale flag

Endpoint Group	Test Cases
Project Members	Invite/remove members, update permission tiers, Developer role changes, permissions enforced

Backend Worker Tests

Framework: Mocha + Chai with mocked BullMQ/Redis/PostgreSQL clients.

Test Area	Test Cases
Queue connection	Worker connects to BullMQ using <code>REDIS_URL</code> , handles missing queue configuration
Job lifecycle	Claims queued jobs, updates progress, marks completion/failure
Persistence	Writes snapshot/job status through PostgreSQL client boundaries
Reliability	Retries failed jobs with bounded retry policy, shuts down cleanly

Analysis Pipeline Tests

Framework: Mocha + Chai with fixture repositories.

Repository Fixtures (5 TypeScript project shapes):

1. Small Express API
2. React frontend app
3. Chrome extension
4. Worker/job processor
5. Library/package

Test Area	Test Cases
Repo inventory	Detects correct app type, finds configs, finds scripts, finds schemas, finds tests
AST extraction	Extracts imports/exports, extracts symbols (functions, classes, interfaces), detects callbacks, detects call expressions, handles circular imports
Code evidence model	Import edges correct, call edges correct, inheritance edges correct, doc links correct, test links correct
Workflow extraction	HTTP route candidate path extracted, extension event candidate path extracted, worker job candidate path extracted, DB write detected as side effect
Workflow collapsing	Noisy internal calls collapsed, onboarding-level path preserved
Privacy filtering	<code>.env</code> files excluded, custom ignored paths excluded, secret patterns filtered

Test Area	Test Cases
Config scanning	<code>package.json</code> , <code>tsconfig.json</code> , Docker, CI, and <code>.env.example</code> facts are extracted without TypeScript AST parsing

Ranking Algorithm Tests

Framework: Mocha + Chai with known inputs and expected outputs.

Test Cases
Auth middleware ranks high (near entrypoints, security side effects)
DB write workflow ranks high (source-of-truth data changes)
Health check ranks lower for backend role, higher for DevOps role
High-churn isolated config does not dominate ranking
Composite score matches manual calculation from individual signal weights
Ranking reasons are generated and human-readable
Role-based re-ranking changes top items appropriately

AI Generation Tests

Framework: Mocha + Chai with mocked OpenRouter/OpenAI-compatible responses.

Test Cases
Evidence bundle sent to OpenRouter contains only extracted structure, not full files
Generated explanation includes source receipts
Uncited claims assigned low confidence or rejected
Generated content has draft review status by default
Regeneration uses updated evidence bundle
AI-disabled mode produces package without AI explanations (deterministic sections only)
Test for prompt injection from repo
Handling various failures, token limits

Incremental Re-analysis Tests

Framework: Mocha + Chai with snapshot fixtures.

Test Cases
Changed file detected via hash comparison

Test Cases
Changed symbol within unchanged file detected via symbol hash
Impacted workflow identified when a step's node changed
Generated package section marked stale when linked workflow is impacted
Existing repo docs are not treated as MVP stale-detection targets
Unchanged files/sections not flagged
Change in schema not handled

5. Task Assignments

Team Member	Role	Main Responsibilities
Nam	API / DevOps / DB / Security	Backend API routes, project CRUD APIs, GitHub OAuth integration, Supabase schema and migrations, API/worker job status persistence, Docker and CI/CD pipeline, deployment, privacy settings, access control, and permission enforcement
Bradley	Worker / AST Parser / Core Analysis	Backend worker, repo ingestion and temporary archive extraction, TypeScript AST parsing with Compiler API, symbol extraction, code evidence model construction, entrypoint and side-effect detection, workflow extraction foundations, snapshot persistence
Sahib	Project Pages / Ranking / Onboarding UX	Per-project pages: Project Overview, Your Onboarding, Architecture Map, and project-scoped dependency/walkthrough views; workflow classification and scoring; Critical 25% / critical path identification; role-based path generation; onboarding package planning; UI/UX for non-trivial project flows
Eugene	App Pages / Team Flows / QA	Intro/landing page, main project list page, import page, tutorials entry page from project pages, Team page, supporting standard frontend pages, frontend component tests, E2E tests with Playwright, and QA support across API and UI

Work Pairing

Pair	Focus
Nam + Eugene	Standard/trivial product surface, API support, team/import/main pages, DevOps, integration, and QA
Bradley + Sahib	Non-trivial core: worker, AST parser, code evidence model, workflow extraction, Critical 25%, role onboarding, dependency graphs, and per-project onboarding pages